

Chains: Indexing Every Clique-Core Hierarchy of a Graph in One Partition

Anonymous Author(s)

Abstract

The k -core of a graph counts the neighbours of a vertex. Counting instead the cliques of s vertices that contain it generalizes the k -core to every clique size s . At each size every vertex gets a core value and lies in one community per level from 1 to its core value, and the communities of a size are nested. A community query names a vertex, a size and a level, and the size is chosen at query time. The direct index for every size stores one hierarchy per size. A vertex is then stored once per size, and the index reaches 190 MB on a web graph of 129,632 vertices. This paper stores each vertex once for all sizes. Two vertices are hierarchy-equivalent when no query, for a value or for a community, at any size tells them apart. We prove that this holds exactly when, at every size, the smallest community containing one vertex, the one at its core value, is the smallest community containing the other. The equivalence classes, called chains, partition the vertices, and no partition into larger groups allows an index to answer every query exactly. We propose ChainIndex, which keeps the hierarchy of every size over chains instead of vertices. It numbers the vertices so that every community, at any size and level, is a few ranges of consecutive vertex ids, found in constant time after a climb to the queried level. The values of a size come from removing vertices one at a time, each time one in the fewest cliques among the vertices left. Removing a vertex only lowers counters on the paths of one clique tree. The tree is never edited, and it gives the values and the hierarchy of every size. Once a value reaches its floor, a binomial lower bound, it stays there at every larger size, and the index neither stores it nor, from the next size on, computes it. On 29 graphs from five families, ChainIndex is $1.28\times$ to $48.7\times$ smaller than one hierarchy per size (median $7.0\times$). It finds the ranges of a community in 12 to 65 nanoseconds and lists its vertices about as fast as a memory copy of the answer. It is built for every size $3.1\times$ to $3547\times$ faster than computing the sizes one at a time. Two case studies on graphs whose vertices carry names and categories show the effect of the size. On DBLP the size whose community best matches a ground-truth community changes from query to query. On Amazon the size narrows a community from a store of 157,331 products to a shelf of six albums.

1 Introduction

Graphs model relations between entities, and many graph-mining tasks look for groups of vertices that are densely connected inside [6, 7, 14]. The k -core is a standard answer: the largest subgraph in which every vertex has at least k neighbours [24]. The k -core counts the neighbours of a vertex. Counting instead the cliques of s vertices that contain the vertex generalizes the k -core to a clique size s . This is the $(1, s)$ -nucleus of Sariyüce et al. [22], where the 1 says that cliques are counted around single vertices rather than around edges or larger cliques. We call this model the clique core. Choose a clique size s and a level k , the k of the k -core. The community of a vertex is the largest set containing it with two properties. Every vertex

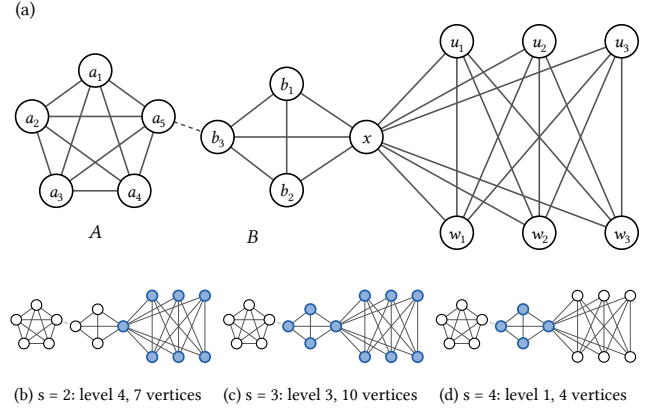


Figure 1: Running example: (a) the graph, with the 5-clique A , the 4-clique $B = \{x, b_1, b_2, b_3\}$, six vertices adjacent to x , and the edge a_5b_3 (dashed), the only edge in no triangle; (b) to (d) the community of x at its core value, for sizes 2, 3 and 4, shaded.

of the set lies in at least k cliques of s vertices inside the set. Any two vertices of the set are joined by a sequence of such cliques in which consecutive cliques share a vertex. We say that the set is connected through these cliques. Size 2 gives the connected pieces of the k -core, since a clique of two vertices is an edge. Size 3 gives the groups held together by triangles. Larger sizes count denser structures. At a given size, a vertex has a community at every level up to a largest level, its *core value* at that size, and at none above it. The communities of one size, nested by level, form the *clique-core hierarchy* of that size.

Example 1.1. Figure 1 shows a 5-clique $A = \{a_1, \dots, a_5\}$, a 4-clique $B = \{x, b_1, b_2, b_3\}$, and a complete bipartite graph between $\{u_1, u_2, u_3\}$ and $\{w_1, w_2, w_3\}$ whose six vertices, written $\{u_1, \dots, w_3\}$, are all adjacent to x . The edge a_5b_3 is the only edge between A and the rest, and it lies in no triangle. At size 2, the core value of x is 4, and its community at level 4 is $\{x, u_1, \dots, w_3\}$, seven vertices with at least four neighbours each inside the set, shaded in Figure 1(b). At size 3 the core value of x is 3 and its community at level 3 is $B \cup \{u_1, \dots, w_3\}$, ten vertices in at least three triangles each inside the set (Figure 1(c)). At size 4 the core value is 1 and the community is B , the one 4-clique through x (Figure 1(d)). The clique A is a community of its own at every size from 2 to 5. Example 2.2 gives the counts.

Applications. The size is the parameter a user changes when the groups of one size are too large or too small. *Collaboration networks.* In a coauthorship graph, the k -core around an author covers a broad research area [18, 19]. The communities at larger sizes keep the authors who repeatedly write papers with several of the same people. *Co-purchase networks.* In a product graph, an edge

joins two products bought together [15]. The community of a jazz album at its core value is, at size 2, a core of more than a hundred thousand products of every kind. At size 3 it is the 23 albums of one record label, at size 4 22 of them, and at size 5 the six albums a shop would place on one shelf (Section 8). Coordinated groups. In a graph whose edges join accounts that like or comment on the same posts, accounts that act together on many posts form cliques, and casual interaction rarely forms one [3, 8]. Larger sizes keep such groups and drop the triangles that arise by chance.

Existing methods and their limitations. A community query names a vertex v , a size s and a level k , and asks for the community of v at (s, k) . A value query asks only for the core value of v at size s . For a fixed size, a community at a higher level lies inside one community at each lower level, and communities at one level are disjoint, so the communities form a tree. A depth-first traversal of that tree writes the vertices into an array, and it writes each vertex when it reaches the smallest community containing the vertex. Every community is then one contiguous block of the array [5, 20]. Existing indexes for community search, that is, for retrieving the community of a query vertex, store one such tree for one model. The model is the k -core [2, 5], the k -truss (the same idea with edges as the unit) [1, 9], or the clique cores of one clique size [20]. An index for every size then holds *one hierarchy per size*. We call this index Baseline and compare against it in the experiments. Call the size of the largest clique containing a vertex its clique number. A vertex lies in cliques of every size from 2 to its clique number, so Baseline stores it in the array of every such size. On the web graph web-uk-2005, cliques reach 500 vertices, a vertex lies in 181 arrays on average, and Baseline takes 190 MB for 129,632 vertices, or 1.5 KB per vertex. The index of this paper stores the same hierarchies in 3.9 MB (Section 8). The alternative is to keep no index and to compute the core values of the queried size when the query arrives, a *decomposition* of the graph. That costs seconds to minutes per query on such graphs. The counting-based decomposition CND [32] needs 15.7 hours to cover the 499 sizes of web-uk-2005 once.

Research question. This leads to the question this paper answers: *how to index the clique-core hierarchies of every size at once, storing each vertex once and returning every community as a few ranges of consecutive ids?*

Challenges. For one size, the tree of that size meets both requirements. Across sizes, the question raises three difficulties. Repetition across sizes. Within one size, two vertices that lie in the same community at every level are interchangeable and could share one record. But the communities of a vertex change from size to size: two vertices that share every community at size 2 may lie in different communities at size 3. The first difficulty is to decide which vertices can share one record for every size, every level and every query without losing an answer. Contiguity of communities. Once vertices are grouped, each group is stored once, at one position of the array. But the depth-first order of one size lists the groups in one order and that of another size in another, so a community, a set of groups, is not one block at every size. The second difficulty is to return any community of any size as a few ranges of vertex ids, in constant time per range. Construction of every size. The tree of one size is the output of one decomposition of the graph. CND counts cliques on clique paths and edits the paths as vertices leave, so the next size starts again from the graph. But the sizes are many,

cliques reach hundreds of vertices, and the decompositions of all sizes together cost hours. The third difficulty is to build the trees of all sizes from one computation.

Our idea. We start from the queries and ask which pairs of vertices no query can tell apart. Chains. Two vertices are *hierarchy-equivalent* when they have the same core value at every size and lie in the same community at every size and level. The *own node* of a vertex at a size is its community at its core value, the smallest community containing it at that size. We prove that two vertices are hierarchy-equivalent exactly when they have the same own node at every size (Theorem 4.3). We call the classes of hierarchy-equivalent vertices *chains*, since a class follows one chain of own nodes through the sizes. Every community is a union of chains, and no partition into larger groups answers every query (Proposition 4.6). On com-dblp, 13,459 chains hold its 317,080 vertices, so one record serves 24 vertices on average. The values repeat across sizes as well. At every size s , the core value of a vertex is at least the number of s -cliques through it inside a largest clique containing it, a binomial coefficient of its clique number that we call its *floor*. Once the value equals the floor at one size, it does so at every larger size (Theorem 5.6), and a chain stores its values only below that size, its *tail start*. Labels, order and runs. We give the vertices new ids, their *labels*, so that every chain is one interval. We order the chains by the depth-first positions of their own nodes, comparing the positions at size 2 first and breaking ties at size 3, 4 and so on. Every size-2 community is then one range of labels (Theorem 5.1), and at larger sizes the same order keeps the number of ranges small (Figure 6). For each size, the index stores the tree of that size, with the same nodes as in Baseline and with every chain at its own node. It also stores the order in which a depth-first traversal of the tree lists the chains, and consecutive labels in this order are merged into *runs*. Where Baseline keeps, for every vertex of every array, its position and a pointer to its own node, the index keeps the runs and one own node per chain and size. A community query climbs the tree from the own node of the vertex to the node of the queried level and reads that node’s ranges in constant time (Theorem 5.3). One static clique tree. A clique tree [11] is a set of paths, each with a mandatory and an optional vertex set, that represents every clique of every size exactly once. The values of one size come from a *peel*. Vertices are removed one at a time, each time one with the smallest clique count among the vertices left, and the largest count seen so far at a removal is the core value of the removed vertex. Removing a vertex never changes a path: the paths on which the vertex is mandatory stop counting, and on the others a counter of the optional vertices left drops by one (Lemma 7.3). The clique tree is therefore *static*, and the same tree serves the peel of every size and gives the tree of every size (Lemma 7.6). The floor saves work here too. A vertex whose value is on the floor at the previous size keeps it. The peel therefore counts cliques for a vertex only up to its tail start: at the sizes whose values the index stores, and at the one size where the value first reaches the floor (Theorem 7.4).

Contributions. Theorem 4.3 decides hierarchy equivalence from the own nodes alone, and Proposition 4.6 shows that the chains are the coarsest partition that loses no answer. ChainIndex stores each vertex once and returns every community as label ranges in constant time after the climb (Theorems 5.1, 5.3 and 5.6). Build computes it for every size from one static clique tree, which gives

the values and the tree of every size (Lemmas 7.3 and 7.6), and the floor spares the peel every vertex whose value at the previous size is already on it (Theorems 5.6 and 7.4). On 29 graphs from five families, ChainIndex is $1.28\times$ to $48.7\times$ smaller than Baseline (median $7.0\times$). It finds the ranges of a community in 12 to 65 nanoseconds, and listing the vertices takes 0.35 to 8.1 times as long as a memory copy of the answer (median 0.99). It is built for every size $3.1\times$ to $3547\times$ faster than one decomposition per size (median $24\times$). Two case studies on graphs whose vertices carry names and categories show the effect of the size. On com-dblp the size whose community best matches a ground-truth community, by F1 score, changes from query to query, and on the product graph com-amazon the size narrows a community from a store to a shelf.

Organization. Section 2 gives the definitions and Section 3 the cost of one hierarchy per size. Sections 4 to 7 give the chains, the index, its queries and its construction, Section 8 the experiments and Section 9 related work.

2 Preliminaries

Graphs and cliques. We consider a simple undirected graph G with vertex set V and n vertices. An s -clique is a set of s pairwise adjacent vertices. The *clique number* of a vertex v , denoted as $\omega(v)$, is the largest size of a clique containing v , and an isolated vertex has clique number 1. A subset of a clique is a clique, so v lies in an s -clique for every size s from 2 to $\omega(v)$ and for no larger size. In Figure 1, $\omega(a_1) = 5$, since A is the largest clique of the graph, and $\omega(x) = 4$, since B is a 4-clique and no two of $u_1, \dots, w_3$ on the same side are adjacent. Also $\omega(u_1) = 3$: u_1 lies in the triangles $\{x, u_1, w_j\}$ and in no 4-clique.

Clique cores. The community model of this paper is the k -core with cliques of s vertices in place of edges, which we call the *clique core* at clique size s and level k and write (s, k) -core.

Definition 2.1 ((s, k)-core). For a clique size $s \geq 2$ and a level $k \geq 1$, an (s, k) -core is a *maximal* set $N \subseteq V$ with two properties: a set with the properties that is contained in no larger set with the properties. Every vertex of N lies in at least k cliques of s vertices contained in N . Any two vertices of N are joined by a sequence of s -cliques contained in N . The first clique contains one of the two vertices, the last contains the other, and consecutive cliques share a vertex. We say that N is connected through its s -cliques.

An (s, k) -core is a k -(1, s)-nucleus in the terminology of Sariyüce et al. [22]. For $s = 2$ the cliques are edges, and a $(2, k)$ -core is a connected k -core: a maximal connected subgraph in which every vertex has at least k neighbours. Two cores of the same size s and level k are disjoint. If they shared a vertex, their union would have both properties, since every vertex keeps its k cliques and a clique sequence inside one core continues through the shared vertex into the other. By maximality each core would then equal the union. The *core value* of v at size s , denoted as $\kappa_s(v)$, is the largest k such that v belongs to an (s, k) -core. We set it to 0 if v lies in no s -clique. Otherwise it is at least 1, since an s -clique containing v has the two properties at level 1 and, the graph being finite, lies inside a maximal such set, an $(s, 1)$ -core. The *community* of v at (s, k) , for $1 \leq k \leq \kappa_s(v)$, is the (s, k) -core containing v . It exists, since the $(s, \kappa_s(v))$ -core containing v has the two properties at level k and

lies inside a maximal such set. It is unique, since cores of the same size and level are disjoint.

Example 2.2. Consider Figure 1 at size 3, writing $\{u_1, \dots, w_3\}$ for the six vertices $u_1, u_2, u_3, w_1, w_2, w_3$ as in Example 1.1. Each a_i lies in $\binom{4}{2} = 6$ triangles, one per pair of the other four vertices of A , and in no other, since a_5 and b_3 have no common neighbour. A is connected through its triangles and no other vertex lies in a triangle with two of its vertices, so A is a $(3, 6)$ -core and $\kappa_3(a_i) = 6$. Each b_i lies in three of the four triangles of B , and each u_i in the three triangles $\{u_i, w_j, x\}$ for $j = 1, 2, 3$, and likewise each w_j . The vertex x lies in twelve triangles, three in B and nine with a u_i and a w_j . The set $N = B \cup \{u_1, \dots, w_3\}$ is a $(3, 3)$ -core. Every vertex of N lies in at least three of its triangles, and N is connected through them, since every vertex of N shares a triangle with x . No vertex of A can be added, since a_5 shares no triangle with N . No vertex of N has core value 4: a $(3, 4)$ -core contains none of $u_1, \dots, w_3$, and without them a vertex of B lies in three triangles only. So $\kappa_3(v) = 3$ on N . At size 2, $\kappa_2(v) = 4$ on A and on $\{x, u_1, \dots, w_3\}$ and $\kappa_2(b_i) = 3$. Every vertex has at least three neighbours, and only x and a_5 have five or more, so no 5-core exists. The vertices b_1 and b_2 have three neighbours, so no 4-core contains them, and without them b_3 has two neighbours left. At size 4, $\kappa_4(a_i) = \binom{4}{3} = 4$ and $\kappa_4(v) = 1$ on B . At size 5, $\kappa_5(a_i) = 1$.

Problem statement. Given a graph G , we aim to build an index that answers two queries for every clique size $s \geq 2$. The *value query* (v, s) returns $\kappa_s(v)$, and the *community query* (v, s, k) with $1 \leq k \leq \kappa_s(v)$ returns the vertex set of the community of v at (s, k) . The index should store each vertex once and return every community as a few ranges of consecutive vertex ids.

3 One Hierarchy per Size

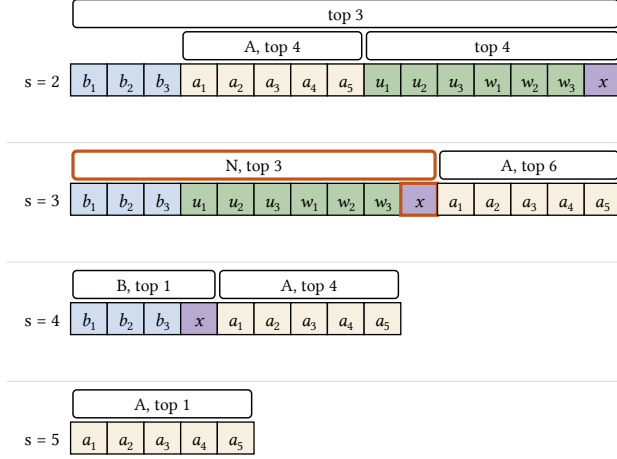
This section describes the hierarchy of one size, that is, the forest that the cores of that size form under inclusion, and the index that stores one such forest per size.

The tree of one size. Fix a size s . Two cores of size s are disjoint or nested. If an (s, k) -core K and an (s, k') -core K' with $k \leq k'$ share a vertex, their union has the two properties of Definition 2.1 at level k , so it is K by maximality, and $K' \subseteq K$. One vertex set can be a core at several levels.

Definition 3.1 (canonical node, own node). A *canonical node* of size s is a vertex set that is an (s, k) -core for at least one level k . Its *top* is the largest such k . The canonical nodes of size s form the forest T_s under inclusion, which we also call the tree of size s although it may have several roots. The nodes contained in no other node are its roots, and the children of a node are the largest nodes strictly inside it. The *own node* of a vertex v with $\kappa_s(v) \geq 1$, denoted as $X_s(v)$, is the $(s, \kappa_s(v))$ -core containing v .

Nesting and maximality give the following facts [22]. The top of a node is the smallest core value of its vertices. Every vertex has a core value of at least the top. If all had more, the node would have the two properties one level higher and would be a core there. A larger set with the properties one level higher would have them at the top as well, against maximality. In particular the top of $X_s(v)$ is $\kappa_s(v)$. Tops increase strictly from a root down to its descendants. A child whose top is at most the top of its parent lies in a larger set

(a) Baseline



(b) ChainIndex

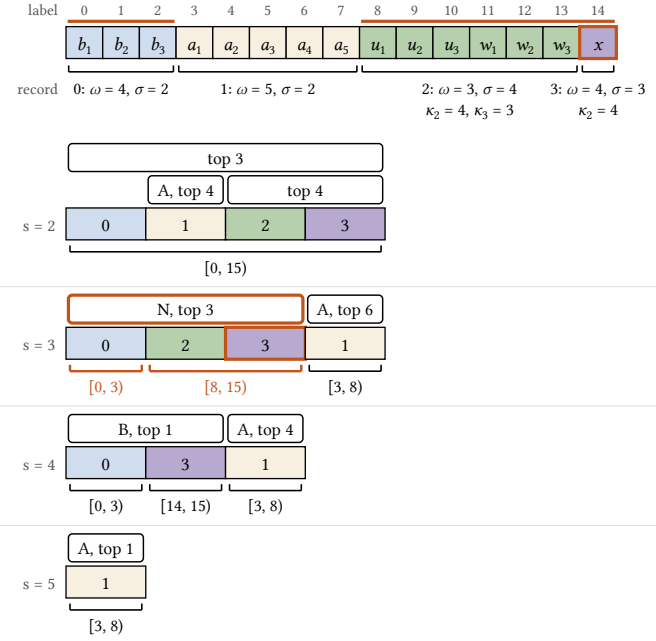


Figure 2: Baseline (a) and ChainIndex (b) for Figure 1, one row per size; tints mark chains, orange the query of x at $(3, 2)$.

with the properties at that level, its parent, against maximality. The nodes containing v form a path from a root down to $X_s(v)$, since every node containing v is a core at a level of at most $\kappa_s(v)$ and so contains $X_s(v)$. For $1 \leq k \leq \kappa_s(v)$, the community of v at (s, k) is the node nearest the root with top at least k on this path. A node nearer the root with top at least k would have the two properties at level k and strictly contain the community. Later sections refer to the community by this description.

Example 3.2. Consider Figure 1, which has cliques of the sizes 2 to 5. Figure 2(a) draws the tree of each size, one bar per node over the array of the size. At size 2 the root holds all 15 vertices with top 3, since every vertex has at least three neighbours and b_1 exactly three. It has two children of top 4, A and $\{x, u_1, \dots, w_3\}$, the two 4-cores of Example 2.2. At size 3 the forest has two roots and no other node, $N = B \cup \{u_1, \dots, w_3\}$ with top 3 and A with top 6. No triangle joins them, and A is the $(3, k)$ -core for every k from 1 to 6. At size 4 the roots are A (top 4) and B (top 1), and at size 5 the forest is A alone. The own node of b_1 is the root at size 2, N at size 3 and B at size 4. The community of x at $(2, 3)$ is the root: both nodes on the path from $X_2(x) = \{x, u_1, \dots, w_3\}$ have top at least 3.

The index of one size. The tree of one size gives an index for that size [5, 20]. The vertices with $\kappa_s(v) \geq 1$ are written into one array by a depth-first traversal of T_s , each vertex when the traversal reaches its own node. The vertices of every node then form one contiguous slice of the array. Every node stores its top, its parent and the two ends of its slice, and every vertex stores its own node and its core value. A community query climbs from the own node to the node nearest the root whose top is at least k and copies that node's slice. A value query reads the stored value.

One hierarchy per size. An index that serves every size stores, for each size, one tree and one array, together with the core values. We call this index Baseline. A vertex v has a nonzero value at every size from 2 to $\omega(v)$ (Section 2), so it appears in $\omega(v) - 1$ arrays, one per size. The arrays therefore hold $\sum_v (\omega(v) - 1)$ positions, 44 for the 15 vertices of Figure 1 (each a_i four times, each b_i and x three times, each u_i and w_j twice), the cells of Figure 2(a). For every such pair of a vertex and a size, Baseline stores the array position and a pointer to the own node. It stores the core values of every vertex in the form that Section 5.4 gives for a chain: the clique number, the tail start and the values below it. Building Baseline means one decomposition per size, 499 of them on web-uk-2005 (Section 1). The next sections keep these trees and store each vertex once for all sizes, at the price of answering a community query with a few ranges of ids instead of one slice.

4 Chains

This section answers the first difficulty of Section 1: which vertices an index can store as one group, with one record for all of them, for every size, every level and every query. The own nodes alone decide hierarchy equivalence (Theorem 4.3), and its classes, the chains, form the coarsest partition of the vertices that loses no answer (Proposition 4.6).

Hierarchy equivalence. Two vertices can share one record when no query tells them apart.

Definition 4.1 (hierarchy equivalence). Two vertices u and v are *hierarchy-equivalent* if, for every size s , their core values agree, $\kappa_s(u) = \kappa_s(v)$, and for every level $1 \leq k \leq \kappa_s(u)$ the two vertices lie in the same (s, k) -core.

Definition 4.2 (trajectory, chain). For a vertex v in some edge, that is, with $\omega(v) \geq 2$, the *trajectory* of v is the tuple of its own nodes over the sizes 2 to $\omega(v)$. We write $\text{traj}(v) = (X_2(v), \dots, X_{\omega(v)}(v))$. The vertices with equal trajectories form a *chain*, one chain per distinct trajectory, and the isolated vertices, those in no edge, form one more chain.

THEOREM 4.3 (OWN NODES DECIDE HIERARCHY EQUIVALENCE). *Two vertices u and v with $\omega(u), \omega(v) \geq 2$ are hierarchy-equivalent if and only if $\text{traj}(u) = \text{traj}(v)$.*

PROOF. ($\Leftarrow$) Equal trajectories have equal length, which gives $\omega(u) = \omega(v)$, and both core values are 0 above that size (Section 2). For $2 \leq s \leq \omega(u)$ the two own nodes are equal, and the top of an own node is the core value of its vertex (Section 3), so $\kappa_s(u) = \kappa_s(v)$. For $k \leq \kappa_s(u)$, the (s, k) -core containing u is the node nearest the root with top at least k on the path from $X_s(u)$ to its root (Section 3). Since $X_s(v) = X_s(u)$, the path and the node are the same for v . ($\Rightarrow$) Equal core values at every size give $\omega(u) = \omega(v)$, since $\omega(v)$ is the last size with a nonzero value. For $2 \leq s \leq \omega(u)$, let $k = \kappa_s(u) = \kappa_s(v)$. The (s, k) -core containing u is $X_s(u)$ by definition. It contains v by hierarchy equivalence, and it is then the $(s, \kappa_s(v))$ -core containing v , that is $X_s(v)$. $\square$

Example 4.4. Consider Figure 1 with the own nodes of Example 3.2, the lowest bar above each vertex in Figure 2(a). The trajectories give four chains: $\{b_1, b_2, b_3\}$ with (root, N , B) over sizes 2 to 4, $\{a_1, \dots, a_5\}$ with (A , A , A , A) over sizes 2 to 5, $\{u_1, \dots, w_3\}$ with $(\{x, u_1, \dots, w_3\}, N)$ over sizes 2 and 3, and $\{x\}$ with $(\{x, u_1, \dots, w_3\}, N, B)$. Figure 2 gives the vertices of each chain one tint. The vertex a_5 has one more edge than a_1 but the same own node at every size, and no query separates them. The vertex x shares the own nodes of u_1 at sizes 2 and 3 and separates from it at size 4, where x lies in the 4-clique B and u_1 in none.

Communities and chains. Every community is a union of whole chains, and every partition with this property has each of its groups inside one chain.

LEMMA 4.5 (COMMUNITIES ARE UNIONS OF CHAINS). *If u lies in an (s, k) -core K with $k \geq 1$ and w is in the chain of u , then $w \in K$.*

PROOF. Since $k \geq 1$, u lies in an s -clique, so its chain is the class of its trajectory, and $\text{traj}(w) = \text{traj}(u)$ gives $X_s(w) = X_s(u)$. Since u lies in K , $\kappa_s(u) \geq k$. The node $X_s(u)$ is a core at the level $\kappa_s(u) \geq k$ that contains u , so it lies inside the (s, k) -core containing u , namely K , by the nesting of Section 3. The node $X_s(u) = X_s(w)$ contains w , and hence $w \in K$. $\square$

PROPOSITION 4.6 (THE COARSEST LOSSLESS PARTITION). *Call a partition of V lossless if every community of every size and level is a union of its blocks, the sets of the partition. The chain partition is lossless, and every lossless partition is at least as fine: each of its blocks lies inside one chain.*

PROOF. The chain partition is lossless by Lemma 4.5. Let P be lossless, and let u and v share a block of P . We show that they share a chain. If $\kappa_s(u) > \kappa_s(v)$ at some size, the community of u at $(s, \kappa_s(u))$ contains u and not v , since every vertex of a core at that level has at least that core value. It is then not a union of blocks

of P . The core values therefore agree at every size. If u is isolated, so is v , and both lie in the chain of isolated vertices. Otherwise, if $X_s(u) \neq X_s(v)$ at a size where both have value $k \geq 1$, these two (s, k) -cores are disjoint (Section 2), and the community of u at (s, k) again contains u and not v . Hence the own nodes agree at every size, $\text{traj}(u) = \text{traj}(v)$, and u and v share a chain. $\square$

Twins. Two vertices u and w are *twins* when their closed neighbourhoods, the neighbours together with the vertex itself, are the same set, and a twin class is a maximal set of pairwise twins. Twins lie in one chain. Swapping them maps G onto itself and every core containing one of them onto a core containing the other. The two cores are the same, since they share a vertex or are the pair itself. Twins are therefore hierarchy-equivalent, and they are adjacent, so Theorem 4.3 puts them in one chain. Chains are therefore at most as many as twin classes, and on the graphs of Section 8 they are far fewer. Twin classes have 1.03 to 1.28 vertices on average, chains 1.5 to 154 (Table 1): com-dblp has 13,459 chains for 317,080 vertices (24 per chain) and web-uk-2005 839 chains for 129,632 vertices (154 per chain).

5 The Chain Index

This section answers the second difficulty of Section 1: how to return any community of any size as a few ranges of labels. The index has three parts, drawn in Figure 2(b) for the running example. They are the vertex-to-chain map, one record per chain, and for every size one tree over the chains with its depth-first array stored as runs of consecutive labels (Section 5.3). We call the tree of a size with its runs the *layer* of that size. In Figure 2(b) the map is the row of labels on top, and the record of each chain is printed under it after the chain number, all but the trajectory, which the rows show. Each layer is one row: the tree as bars over the chains, and the runs as brackets under them with their label ranges. The tree of a size is the tree T_s of Baseline, with the same nodes. What the index replaces is what Baseline stores per vertex: a position and a pointer to the own node in every array, and the values (Section 5.5).

5.1 Aligned labels

Chains partition V (Section 4), so the vertices can be relabelled so that every chain is one interval of labels. The C chains are numbered 0 to $C - 1$ in the order of Section 5.2. The interval of a chain runs from its first label to the first label of the next chain, or to n for the last chain. Call the first label of each chain a *chain start*. The chain of v , written $\text{chain}(v)$, is the number of chain starts at or before the label of v , minus one. This is a rank query over the n labels with the C chain starts marked, and it takes constant time with $n + o(n)$ bits [10]. The map stores those bits and the first label of every chain. In Figure 2(b) the chain starts are 0, 3, 8 and 14, and u_1 , with label 8, has three starts at or before it and lies in chain 2. A community is a union of chains (Lemma 4.5) and so a list of label ranges. If the graph is kept under its original ids, the index also stores the permutation from ids to labels, which is n labels long.

5.2 Chain order

The vertices of a chain c share their own node at every size, written $X_s(c)$. Fix any preorder numbering pre_s of each forest T_s and

give chain c the key $(\text{pre}_2(X_2(c)), \text{pre}_3(X_3(c)), \dots)$, the preorder positions of its own nodes size by size. Chains are ranked in lexicographic key order, a key that is a prefix of another ranking first. The chain of the isolated vertices has an empty key and is placed last.

THEOREM 5.1 (SINGLE RANGES AT SIZE 2). *Under the lexicographic order, the labels of every $(2, k)$ -core form one interval.*

PROOF. An $(2, k)$ -core is the vertex set of the subtree of T_2 rooted at some node X . A chain c lies in it if and only if its own node $X_2(c)$, the deepest node containing c , is in that subtree. The preorder numbers of a subtree form an interval, and the chains whose first key component lies in an interval are contiguous in lexicographic order. Contiguous ranks are contiguous labels. $\square$

Example 5.2. Number the bars of each row of Figure 2(b) by their left ends, which gives a preorder of the tree of that size. The own node of a chain is the lowest bar above it, so the keys of the chains $\{b_1, b_2, b_3\}$, $\{a_1, \dots, a_5\}$, $\{u_1, \dots, w_3\}$ and $\{x\}$ are $(0, 0, 0)$, $(1, 1, 1, 0)$, $(2, 0)$ and $(2, 0, 0)$. This gives the chains 0 to 3, and the $(2, 4)$ -core $\{x, u_1, \dots, w_3\}$ is the interval $[8, 15)$.

The size-2 layer of the index is therefore an index for connected k -core communities in which every community is one label range.

5.3 Run arrays and entry points

A chain is *active* at size s when s is at most its clique number. The *depth-first array* of size s lists the active chains in a depth-first traversal of T_s . At each node, the traversal has two kinds of items to list: the chains whose own node it is, and the subtrees of its children. It lists them in one sequence in increasing order of the smallest chain rank each item contains, a chain counting as its own rank, and it orders the roots in the same way. The chains of any subtree are then one contiguous segment of the array, starting with the chain of smallest rank in the subtree. The order in which the traversal visits the nodes is a preorder of T_s , the *traversal order*, in general different from pre_s . For $s = 2$ the array is the rank order itself, since the first key component places the chains of a node before its subtrees and the subtrees in the order of pre_2 . When the chain listed after a chain of rank r has rank $r + 1$, the label intervals of the two are adjacent, the second following the first. Maximal sequences of such chains are merged into *runs*, numbered 0 to $R - 1$ where R is their number, and run r covers the labels from $\text{lo}(r)$ to $\text{hi}(r)$, the latter excluded. Every node X stores one *entry* (r_X, v_X) : the number of the run that holds the first label of the segment of X , and that label. A sentinel entry $(R, \text{hi}(R - 1))$ follows the last node.

THEOREM 5.3 (RETRIEVAL). *Let Y be the node that follows the subtree of X in the traversal order, and let (r_Y, v_Y) be its entry, or the sentinel entry if no node follows. If $r_X = r_Y$, the labels of the vertices of X are the one range $[v_X, v_Y)$. Otherwise they are the first range $[v_X, \text{hi}(r_X))$, the whole runs $r_X + 1, \dots, r_Y - 1$, and the last range $[\text{lo}(r_Y), v_Y)$. When Y is the sentinel there is no last range. The number of nonempty ranges equals the number of runs that overlap the segment, and at $s = 2$ it is one.*

PROOF. The segment of X in the depth-first array starts at the first chain of X . It ends before the first chain of Y , or at the end

of the array when Y is the sentinel. Runs are maximal over the whole array, so inside the segment a run can be cut only at the two ends of the segment. The run r_X contributes its labels from v_X on, and the runs strictly between r_X and r_Y are whole. The run r_Y contributes its labels before v_Y , an empty range when the segment ends at a run boundary. If Y is the sentinel, the segment ends with run $R - 1$, which is whole. Each run that overlaps the segment gives one range, and at $s = 2$ the whole array is one run, since it is the rank order. $\square$

Example 5.4. Consider the row $s = 3$ of Figure 2(b). The two roots N and A have no children. The traversal lists N before A , since N holds chain 0, and the chains of N in rank order, so the depth-first array is chain 0, chain 2, chain 3, chain 1. Chains 2 and 3 have consecutive ranks and merge into one run, so the array is stored as the three runs $[0, 3)$, $[8, 15)$ and $[3, 8)$, the brackets under the row, numbered 0 to 2 from left to right. The entries are $N \rightarrow (0, 0)$, $A \rightarrow (2, 3)$ and the sentinel $(3, 8)$. The community of x at $(3, 2)$, traced in orange in Figure 2, is N , followed by A with $r_N = 0 \neq r_A = 2$. The answer is the first range $[0, 3)$, the whole run $[8, 15)$ and the empty last range $[\text{lo}(2), 3) = [3, 3)$: two ranges for ten vertices. The community A at $(3, 6)$ is followed by the sentinel, $r_A = 2 \neq 3$, and is the first range $[3, \text{hi}(2)) = [3, 8)$ alone.

5.4 Values

Core values are constant on a chain (Theorem 4.3), and so is the clique number, the last size with a nonzero value. Inside a largest clique containing it, a vertex lies in $\binom{\omega(v)-1}{s-1}$ s -cliques, and the clique has the two properties of Definition 2.1 at that level. The value of the vertex at size s therefore cannot fall below this number, its *floor* at size s . The next theorem says that once the value reaches the floor at some size, it stays on the floor at every larger size, so the values from that size on need not be stored. Its proof uses the Kruskal–Katona theorem in Lovász’s form [12, 13, 16], with the binomial coefficient $\binom{t}{s} = t(t-1) \cdots (t-s+1)/s!$ extended to real t . For $t \geq s - 1$ both $\binom{t}{s}$ and $\binom{t}{s-1}$ increase strictly with t .

LEMMA 5.5 (KRUSKAL–KATONA). *For $m \geq 1$ let $t \geq s$ be the real number with $\binom{t}{s} = m$. Any m or more distinct s -element sets have at least $g_s(m) = \binom{t}{s-1}$ distinct $(s-1)$ -element subsets.*

THEOREM 5.6 (FLOOR AND TAIL). *Let $2 \leq s \leq \omega(v)$.*

- (a) $\kappa_s(v) \geq \binom{\omega(v)-1}{s-1}$.
- (b) If $\kappa_s(v) = \binom{\omega(v)-1}{s-1}$ and $s < \omega(v)$, then $\kappa_{s+1}(v) = \binom{\omega(v)-1}{s}$.

PROOF. (a) A largest clique L containing v has $\omega(v)$ vertices. Every vertex of L lies in $\binom{\omega(v)-1}{s-1}$ s -cliques of L , and L is connected through them. So L has the two properties of Definition 2.1 at the level of the floor and lies inside a core of size s at that level.

(b) Let K be the $(s+1, \kappa_{s+1}(v))$ -core containing v , and take any vertex u of K . It lies in at least $\kappa_{s+1}(v)$ cliques of $s+1$ vertices inside K , and removing u from them leaves as many distinct s -element sets of neighbours of u in K . By Lemma 5.5 these sets have at least $g_s(\kappa_{s+1}(v))$ distinct $(s-1)$ -element subsets, and each of them forms with u an s -clique of K . Every vertex of K hence lies in at least $g_s(\kappa_{s+1}(v))$ s -cliques of K . K is connected through these s -cliques, because two $(s+1)$ -cliques that share a vertex contain s -cliques that share it. So K has the two properties at size s and level $g_s(\kappa_{s+1}(v))$,

lies inside a core at that level, and $\kappa_s(v) \geq g_s(\kappa_{s+1}(v))$. If $\kappa_{s+1}(v) > \binom{\omega(v)-1}{s}$, then the t with $\binom{t}{s} = \kappa_{s+1}(v)$ exceeds $\omega(v) - 1 \geq s$, so $\kappa_s(v) \geq \binom{t}{s-1} > \binom{\omega(v)-1}{s-1}$, contradicting the assumption of (b). Since $\kappa_{s+1}(v) \geq \binom{\omega(v)-1}{s}$ by (a), equality holds. $\square$

We write $\sigma(v)$, the *tail start*, for the smallest size at which the value equals the floor, and we set $\sigma(v) = \omega(v) + 1$ when no size does. For $s \geq \sigma(v)$ the value is $\binom{\omega(v)-1}{s-1}$, by (b) applied size after size, and is not stored. The values for $2 \leq s < \sigma(v)$ are the *residue* of v .

Example 5.7. In Figure 1, $\kappa_2(a_i) = 4 = \binom{4}{1}$ with $\omega(a_i) = 5$ and $\kappa_2(b_i) = 3 = \binom{3}{1}$ with $\omega(b_i) = 4$, so $\sigma(a_i) = \sigma(b_i) = 2$ and no value of the chains $\{a_1, \dots, a_5\}$ and $\{b_1, b_2, b_3\}$ is stored. The vertex x has $\kappa_2(x) = 4 > \binom{3}{1}$ and $\kappa_3(x) = 3 = \binom{3}{2}$, so $\sigma(x) = 3$ and its residue is the value 4. The six vertices $u_1, \dots, w_3$, with $\omega = 3$, have values 4 and 3 above their floors $\binom{2}{1} = 2$ and $\binom{2}{2} = 1$, so $\sigma = 4$ and both values are residues. Stored per chain, the whole graph keeps three values, the residues under the chains in Figure 2(b).

The record of a chain holds its clique number ω , its tail start σ , its residues, and its trajectory (Definition 4.2). The first three answer every value query without reading a layer (Section 6), and the trajectory starts every community query. A value at or past σ is the binomial coefficient of Theorem 5.6, and for $s > \omega$ it is 0.

5.5 Size

Let a word hold one label, one node or one value. The map takes $n + o(n)$ bits and C words for the first labels. The chain records take $2C$ words for ω and σ , one word per (chain, size) pair for the trajectories, and one word per residue. The layers take a constant number of words per canonical node (parent, jump pointer, top, subtree size and entry) and two labels per run, and the permutation of Section 5.1, if kept, n labels. Baseline takes, for every (vertex, size) pair, one word for the array position and one for the pointer to the own node, and a constant number of words per canonical node. It stores the values in the form of Section 5.4, but per vertex: ω , σ and the residues of every vertex. The canonical nodes are the same in both indexes. The saving is in the pairs and in the values: the array positions become two labels per run, the $\sum_v(\omega(v) - 1)$ pointers become the $\sum_c(\omega(c) - 1)$ own nodes of the trajectories, and ω , σ and the residues are stored once per chain instead of once per vertex. In Figure 2 the 44 array positions of (a) become the 8 runs of (b), the brackets under its rows, and the 44 pointers become one own node per chain cell of (b), 12 in all. Both panels draw the same 8 canonical nodes, and the 13 residues of the vertices, which (a) does not draw, become the 3 residues printed in (b).

6 Queries

Both queries start from the vertex-to-chain map of Section 5.1. The value query then reads the record of the chain (Section 5.4), and the community query climbs the tree of the queried size and reads two entries (Section 5.3).

Algorithm. Algorithm 1 has two procedures, *Value* and *Community*. *Value* reads the chain of v from the map (Line 2) and returns 0 when s exceeds the chain's clique number (Line 3). Based on Theorem 5.6, it returns the floor $\binom{\omega-1}{s-1}$ when s is at or past the tail start (Line 4),

Algorithm 1: Query: value and community queries on ChainIndex.

Input: the index; a vertex v ; a size s ; for a community query, a level k with $1 \leq k \leq \kappa_s(v)$.

Output: $\kappa_s(v)$, or the community of v at (s, k) as label ranges.

```

1 Procedure Value( $v, s$ )
2    $c \leftarrow \text{chain}(v)$ 
3   if  $s > \omega[c]$  then return 0
4   if  $s \geq \sigma[c]$  then return  $\binom{\omega[c]-1}{s-1}$ 
5   return residue[ $c$ ][ $s$ ]
6 Procedure Community( $v, s, k$ )
7    $c \leftarrow \text{chain}(v); X \leftarrow \text{trajectory}[c][s]$ 
8   while parent[ $X$ ]  $\neq \text{none}$  and top[parent[ $X$ ]]  $\geq k$  do
9     if jump[ $X$ ]  $\neq \text{none}$  and top[jump[ $X$ ]]  $\geq k$  then
10       $X \leftarrow \text{jump}[X]$ 
11    else  $X \leftarrow \text{parent}[X]$ 
12  end
13   $Y \leftarrow X + \text{size}[X]$ 
14   $(r_X, v_X) \leftarrow \text{entry}[X]; (r_Y, v_Y) \leftarrow \text{entry}[Y]$ 
15  if  $r_X = r_Y$  then return the range  $[v_X, v_Y]$ 
16  return  $[v_X, \text{hi}(r_X)), [\text{lo}(r), \text{hi}(r))$  for  $r_X < r < r_Y$ ,
17     $[\text{lo}(r_Y), v_Y]$ 

```

and otherwise the stored residue (Line 5). *Community* reads the chain and its own node X at size s from the trajectory (Line 7). It climbs to the node nearest the root whose top is at least k (Lines 8 to 10), the community of v at (s, k) by Section 3. Besides its parent pointer, every node carries a *jump pointer* to an ancestor, chosen as in the skip pointers of Myers [17]. Along every root path, the jump pointers reach any ancestor with $O(\log d)$ jumps and parent steps for a climb of d levels. A jump is taken whenever the jump target still has top at least k (Line 9). This is safe. Tops grow from a root down to the leaves, so every node skipped over has top at least k as well, and none of them is the node nearest the root with that property. Nodes are numbered in the traversal order of Section 5.3 and every node stores its subtree size, so the node after the subtree of X is $X + \text{size}[X]$ (Line 12), the sentinel for the last subtree. Based on Theorem 5.3, the two entries give the answer (Lines 13 to 15). The answer is one range when both entries lie in the same run, and otherwise a first range, whole runs and a last range.

Example 6.1. Consider Figure 2(b). For *Value*($u_1, 3$): the chain of u_1 is 2 (Line 2), with $\omega = 3$ and $\sigma = 4$, since its values 4 and 3 never reach the floors $\binom{2}{1}$ and $\binom{2}{2}$. The answer is the residue at size 3, which is 3 (Line 5). For *Value*($a_2, 4$): the chain is 1 with $\omega = 5$ and $\sigma = 2 \leq 4$, and the answer is the floor $\binom{4}{3} = 4$ (Line 4). For *Community*($x, 3, 2$): the chain is 3 and its trajectory gives the own node N at size 3 (Line 7), a root, so the climb stops at once (Line 8). N has no children, so the node after it is A (Line 12), and the entries (0, 0) and (2, 3) (Line 13) give the two ranges of Example 5.4 (Line 15).

Complexity. *Value* takes constant time: one rank query on the chain starts, the record's ω and σ , and one residue or one binomial coefficient. The coefficients up to the largest clique number are

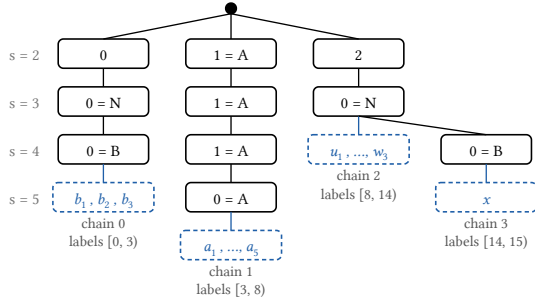


Figure 3: The trie that refines the chains of Figure 1 size by size; the vertices stop at the dashed leaves.

computed once with the index. Community takes $O(\log d)$ steps for a climb of d levels and constant time for the two entries, plus one step per range to report the answer, or one per vertex to list it. No step of either procedure touches the graph.

7 Construction

This section answers the third difficulty of Section 1: how to build the trees of every size from one computation. The values of one size come from a *peel*. The peel removes the vertices one at a time, each time one with the smallest *clique count*, the number of s -cliques through it among the vertices not yet removed. The largest count seen at a removal so far is the core value of the removed vertex. A decomposition that counts cliques on clique paths, such as CND, edits the paths as vertices leave, so every size starts again from the graph. The index is built instead from one clique tree that no step edits: a removal only lowers counters on its paths. The same tree serves the peel of every size, which counts cliques only for the vertices whose values are not yet known, and gives the tree of every size, which splits the chains (Figure 3).

One clique tree for every size. The clique counts of every size come from one pivoting clique tree [11].

Definition 7.1 (clique tree). A *clique tree* of G is a set of paths. A path P carries two disjoint vertex sets, a nonempty *mandatory set* H and an *optional set* Q , whose union is a clique. At size s the path represents the s -cliques $H \cup Z$ for every $Z \subseteq Q$ with $|Z| = s - |H|$, and every s -clique of G is represented by exactly one path.

A path represents cliques at the sizes from $|H|$ to $|H| + |Q|$, so one clique tree serves every size. For a vertex v and a vertex set W , write $c_s(v, W)$ for the clique count of v in W , the number of s -cliques that contain v and lie inside W .

LEMMA 7.2 (CLIQUE COUNT). For $v \in W$, $c_s(v, W)$ is the sum, over the paths through v whose mandatory set lies in W , of $\binom{|Q \cap W| - r}{s - |H| - r}$, where $r = 1$ if $v \in Q$ and $r = 0$ otherwise.

PROOF. An s -clique through v inside W is represented by one path P through v as $H \cup Z$, and it lies in W exactly when $H \subseteq W$ and $Z \subseteq Q \cap W$. If $v \in H$, any $s - |H|$ vertices of $Q \cap W$ form Z , and if $v \in Q$, Z contains v and $s - |H| - 1$ of the other vertices of $Q \cap W$. $\square$

The peel of a size removes vertices from W one at a time, and the next lemma gives the change of a count at a removal.

LEMMA 7.3 (REMOVAL). Let $u \neq v$ lie in W . Then $c_s(u, W) - c_s(u, W \setminus \{v\})$ is the sum, over the paths through both u and v whose mandatory set lies in W , of $\binom{|Q \cap W| - r}{s - |H| - r}$, where r is the number of the two vertices u and v that lie in Q .

PROOF. The cliques counted in $c_s(u, W)$ and not in $c_s(u, W \setminus \{v\})$ are those that contain v as well. By the argument of Lemma 7.2, a path through both vertices represents $\binom{|Q \cap W| - r}{s - |H| - r}$ of them, and a path without v represents none. $\square$

A path thus depends on W only through whether its mandatory set lies in W and through $|Q \cap W|$. Removing a mandatory vertex ends the path's contribution, and removing an optional vertex lowers $|Q \cap W|$ by one. The peel therefore keeps for every path one *counter*, the number $q(P) = |Q \cap W|$ of its optional vertices not yet removed while its mandatory set is intact, and never edits a path, so the clique tree is *static*.

Change-only peel with settled vertices. The peel of a size counts cliques for every vertex, but Theorem 5.6 already gives many of the values that the peel computes. A vertex v with $\omega(v) \geq s$ is *settled* at size s when $s > \sigma(v)$, that is, when its value at size $s - 1$ is already its floor. By Theorem 5.6(b), its value at size s is then the floor $\binom{\omega(v) - 1}{s - 1}$. The other vertices, the *unsettled* ones, are peeled. In the peel below, the *key* of a settled vertex is its value, and the key of an unsettled vertex is its clique count among the vertices not yet removed. At size 2, the first size peeled, no vertex is settled.

THEOREM 7.4 (PEEL WITH SETTLED VERTICES). Let every vertex v with $\omega(v) \geq s$ be either settled, with key $\kappa_s(v)$, or unsettled, with key its clique count. Remove these vertices one at a time, each time one of smallest key, and give each the largest key seen at a removal so far. Every such vertex receives its core value $\kappa_s(v)$.

PROOF. Fix a level $k \geq 1$, and let D_k be the vertices of value at least k , the union of the cores of size s at level k . Every vertex of D_k lies in at least k s -cliques inside D_k . Conversely, a set whose every vertex lies in k or more s -cliques inside it lies inside D_k : each part connected through these cliques has both properties of Definition 2.1 at level k . Until a vertex of D_k is removed, every vertex of D_k has key at least k , since its value, or else its count, is at least k . The first vertex of D_k to be removed thus has key at least k , and a removal with key at least k occurs whenever D_k is not empty. Just before the first removal with key at least k , the remaining vertices, call them R , include all of D_k . Every vertex of R has key at least k , and an unsettled one lies in at least k s -cliques inside R because its key is its count. A settled one has value at least k and lies in D_k , where it lies in at least k s -cliques, all inside R . By the converse, $R = D_k$, so exactly the vertices of D_k receive a value of at least k , and every vertex receives $\kappa_s(v)$. $\square$

By Lemma 7.3, a removal changes only the counts of the vertices that share a path with the removed vertex, and the peel never reads the count of a settled vertex. Procedure Peel of Algorithm 2 is this peel. It keys the settled vertices by their floors, sets the counters, and counts the others by Lemma 7.2 (Lines 3–5). It removes a vertex of smallest key and gives it the largest key seen so far (Lines 7–8). It

then visits the paths through the removed vertex whose mandatory set was inside A (Line 9). On each, it subtracts from every unsettled vertex still in A the loss of Lemma 7.3, the only change the removal causes, so the peel updates only what changes (Line 10). If the removed vertex is optional on the path, the counter drops by one (Line 11), and a path whose mandatory set lost a vertex is never visited again.

Example 7.5. Consider Figure 1 at size 3. The vertices $a_1, \dots, a_5$ and b_1, b_2, b_3 have their floors 4 and 3 as values at size 2 (Example 5.7), and they are settled with the floors $\binom{4}{2} = 6$ and $\binom{3}{2} = 3$. The value of x and of the six vertices $u_1, \dots, w_3$ at size 2 is 4, above their floors 3 and 2, and these seven vertices are unsettled. The peel counts triangles only for them: x lies in 12, nine with one of u_1, u_2, u_3 and one of w_1, w_2, w_3 , and three inside B , and each of the other six lies in 3. The smallest keys are 3, of $u_1, \dots, w_3$ and of the settled b_1, b_2, b_3 , and these nine vertices leave at level 3. Once $u_1, \dots, w_3$ are gone, x lies in at most the three triangles of B , fewer if some b_i left first, and it leaves at level 3 as well. No triangle joins $a_1, \dots, a_5$ to an unsettled vertex, and their removals change no key. At size 4, x is settled, since its value 3 at size 3 equals its floor $\binom{3}{2}$ there, and the a_i and b_i stay settled.

The tree of a size. Once the values of size s are known, the tree T_s is read off the static clique tree without listing a clique. A clique of a path enters the cores at the smallest value among its vertices. For a path with $|H| \leq s \leq |H| + |Q|$, let $\eta = s - |H|$, let h be the smallest value of size s on H , and order Q as $p_1, \dots, p_m$ by nonincreasing value, where $m = |Q|$ and $\kappa_s(p_0) = \infty$. The path makes one *event* for its first clique, $H \cup \{p_1, \dots, p_\eta\}$ at level $\min(h, \kappa_s(p_\eta))$, and, if $\eta \geq 1$, one event $\{p_1, p_i\}$ at level $\min(h, \kappa_s(p_i))$ for every $i > \eta$ (for $\eta = 0$ the only event is H at level h).

LEMMA 7.6 (EVENTS). *For every level $k \geq 1$, joining the vertices of each event of level at least k gives as components exactly the cores of size s at level k .*

PROOF. An event of level at least k joins vertices of value at least k that are connected through s -cliques of such vertices. The first event is a clique, and for $i > \eta$ the vertices p_1 and p_i lie together in the clique $H \cup \{p_1, \dots, p_{\eta-1}, p_i\}$ when $\eta \geq 2$, whose values are at least $\min(h, \kappa_s(p_i))$ by the order of Q , or share the cliques $H \cup \{p_1\}$ and $H \cup \{p_i\}$ when $\eta = 1$. Conversely, let $C = H \cup Z$ be an s -clique of a path whose vertices all have value at least k . Then $h \geq k$, and the η vertices of Z have values at least k , so $\kappa_s(p_\eta) \geq k$ and the first event has level at least k . Every vertex p_i of Z with $i > \eta$ joins p_1 in an event of level $\min(h, \kappa_s(p_i)) \geq k$, so C lies in one component. The components are therefore the parts of the vertices of value at least k connected through their s -cliques, the cores at level k . $\square$

Procedure Hierarchy of Algorithm 2 collects the events (Lines 17–19) and processes them by decreasing level with a union-find structure (Lines 21–22). By Lemma 7.6, after the events of level k every component that changed is a canonical node of top k , and the nodes it absorbed become its children (Line 23). Every vertex of value k receives the node of top k containing it as its own node (Line 24). **Algorithm.** Algorithm 3 builds the clique tree once (Line 1), and by Lemma 7.3 no later step edits it; it starts from one class holding every vertex (Line 2). For every size (Line 3) it computes the values

Algorithm 2: Peel and Hierarchy at size s .

```

1 Procedure Peel( $s, S$ )
2    $A \leftarrow \{v : \omega(v) \geq s\}; \ell \leftarrow 0$ 
3    $\text{key}(v) \leftarrow \binom{\omega(v)-1}{s-1}$ , for every  $v \in S$ 
4    $q(P) \leftarrow |Q \cap A|$ , for every path  $P$ 
5    $\text{key}(v) \leftarrow c_s(v, A)$ , for every  $v \in A \setminus S$ 
6   while  $A \neq \emptyset$  do
7      $v \leftarrow$  a vertex of  $A$  with the smallest key
8      $A \leftarrow A \setminus \{v\}; \ell \leftarrow \max(\ell, \text{key}(v)); \kappa_s(v) \leftarrow \ell$ 
9     foreach path  $P$  through  $v$  with  $H \subseteq A \cup \{v\}$  do
10       $\text{key}(u) \leftarrow \text{key}(u) - \binom{q(P)-r}{s-|H|-r}$ , for
11       $u \in (H \cup Q) \cap A \setminus S, r = |\{u, v\} \cap Q|$ 
12      if  $v \in Q$  then  $q(P) \leftarrow q(P) - 1$ 
13    end
14  return  $\kappa_s(\cdot)$ 
15 Procedure Hierarchy( $s$ )
16   $E \leftarrow \emptyset; T_s \leftarrow \emptyset$ 
17  foreach path with  $|H| \leq s \leq |H| + |Q|$  do
18    add  $H \cup \{p_1, \dots, p_\eta\}$  at level  $\min(h, \kappa_s(p_\eta))$  to  $E$ 
19    if  $\eta \geq 1$  then add  $\{p_1, p_i\}$  at level  $\min(h, \kappa_s(p_i))$  to
20     $E$ , for  $\eta < i \leq m$ 
21  end
22  foreach level  $k \geq 1$  of  $E$ , in decreasing order do
23    join the vertices of every event of level  $k$ 
24    add every component that changed to  $T_s$  as a node
25    of top  $k$ , with the nodes it absorbed as children
26     $X_s(v) \leftarrow$  the node of top  $k$  containing  $v$ , for  $v$  with
     $\kappa_s(v) = k$ 
27  end
28  return  $(T_s, X_s)$ 

```

(Step 1) and the tree with the refinement of the classes (Step 2), and after the last size it lays out the index (Step 3). *During Step 1*, the algorithm collects the vertices settled at size s (Line 4). Based on Theorem 7.4, Peel keys them by their floors and counts cliques only for the other vertices (Line 5). *During Step 2*, Hierarchy builds T_s and the own nodes of size s from the same clique tree (Line 6). The algorithm then splits every class by the own node of size s , where $X_s(v) = \perp$ for a vertex of value 0 (Line 7). After the last size, two vertices share a class exactly when they have the same own node at every size, so by Theorem 4.3 the classes are the chains. Figure 3 draws this refinement as a trie, one level per size. *During Step 3*, the algorithm ranks the chains by the keys of Section 5.2 and gives the vertices their labels (Line 9). It builds the run array and the entries of every size as in Section 5.3 (Line 10). It stores for every chain its clique number, its tail start, its residues and its trajectory, as in Section 5.4 (Line 11).

Example 7.7. In Figure 1 at size 2, one path of the clique tree has $H = \{a_1\}$ and $Q = \{a_2, \dots, a_5\}$, all of value 4. It makes the event $\{a_1, a_2\}$ and the events $\{a_2, a_i\}$ for $i \geq 3$, all at level 4, and these join A . The events of level 4 join $\{x, u_1, \dots, w_3\}$ in the same way, and the

Algorithm 3: Build: the chain index from one clique tree.

Input: a graph G .
Output: ChainIndex.

```

1  $T \leftarrow$  the clique tree of  $G$ 
2  $\mathcal{P} \leftarrow \{V\}$ 
3 for  $s \leftarrow 2, 3, \dots$  while  $G$  has an  $s$ -clique do
    // Step 1: the values of size  $s$ 
4    $S \leftarrow$  the vertices settled at size  $s$ 
5    $\kappa_s \leftarrow \text{Peel}(s, S)$ 
    // Step 2: the tree of size  $s$  and the chains
6    $(T_s, X_s) \leftarrow \text{Hierarchy}(s)$ 
7    $\mathcal{P} \leftarrow \{ \{v \in C : X_s(v) = y\} : C \in \mathcal{P}, y \in T_s \cup \{\perp\} \} \setminus \{\emptyset\}$ 
8 end
    // Step 3: the layout
9   rank the chains  $\mathcal{P}$  by their keys and label the vertices
10 foreach size  $s$  do build the run array of  $T_s$ 
11 foreach chain  $c \in \mathcal{P}$  do store its record

```

two components are the nodes of top 4. The path with $H = \{b_3\}$ and $Q = \{a_5\}$ makes the event $\{b_3, a_5\}$ at level $\min(3, 4) = 3$. The events of level 3 join everything into the root of top 3, and the partition then has three classes, $\{b_1, b_2, b_3\}$, $\{a_1, \dots, a_5\}$ and $\{x, u_1, \dots, w_3\}$. At size 3 no class is split. At size 4 the third class splits: $u_1, \dots, w_3$ have value 0, and x has the own node B . The four classes, the dashed leaves of Figure 3, are the chains of Example 4.4, ranked 0 to 3 by their keys. The layout of size 3 produces the runs of the row $s = 3$ in Figure 2(b).

Cost. Let I_s be the total size of the paths with $|H| \leq s \leq |H| + |Q|$. Peel reads each such path once to count (Lemma 7.2) and once per removed vertex of the path while its mandatory set is intact (Lemma 7.3). Hierarchy makes at most I_s events and at most I_s unions and sorts them in $O(I_s \log I_s)$ time. The refinement adds one step per vertex and size, and the layout one pass per size over the chains. The working space is the clique tree and the graph, plus the values of two consecutive sizes, the partition, and the trees of the sizes with one node per canonical node.

8 Experiments

Hardware. All algorithms are implemented in C++ and run single-threaded on a laptop (Apple M-series) and two Ubuntu 22.04 servers, server 1 and server 2 (GCC 11.4, 96 cores, 503 GB of memory each). We never ran two timing runs at once on one machine. Jobs of other users ran on the servers at the same time, with a load average of 1 to 8 on the 96 cores. One core of the laptop is 1.5 to 3 times faster than one core of a server, so laptop and server latencies are not directly comparable, and every table and figure names its machine. **Algorithms.** ChainIndex is the index of Sections 5 to 7, built by Build and queried by Query. Baseline is the index of Section 3, one tree and one depth-first array per size, with the values of every vertex stored in the form of Section 5.4. Its size and its query latency are measured with the core values, trees and queries used for ChainIndex. CND is the published counting-based nucleus decomposition [32], run once per size with its default settings. It computes

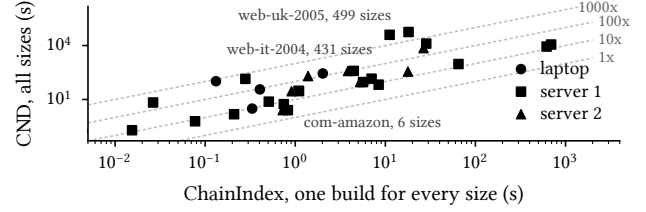


Figure 4: Build time of CND, all sizes, against ChainIndex.

core values only, so it is compared on build time and not on index size.

Datasets. We use 29 real graphs from five families, collaboration, citation, web, product co-purchase, and social and communication networks, with 1,005 to 4,049,537 vertices and largest cliques of 7 to 500 vertices. Table 1 lists them with their family and their largest clique size $s_{\max}$, so the sizes at which some vertex has a nonzero core value are 2 to $s_{\max}$. Three larger graphs run out of the 503 GB of memory of the servers while the clique tree is built, and they are not among the 29: com-lj, ca-hollywood-2009 and com-orkut.

Metrics. Each query draws a vertex that lies in an edge, and then a size s between 2 and the clique number of that vertex. The workload has 20,000 community queries at the vertex’s own level, $k = \kappa_s(v)$, 20,000 at half that level (rounded down, at least 1), 1,000 at $k = 1$, and 200,000 value queries, which ask for a core value. Each workload runs six times, the first as a warm-up. For each of the other five runs we divide the total time by the number of queries, and the latency is the median of these five values. Index sizes are measured in memory, and build times exclude the reading of the graph.

Exp-1: Construction against one decomposition per size. The build time of ChainIndex (Table 1) has three parts: the clique tree, the peel of every size, and the index passes of Section 7 (the trees of the sizes, the partition and the layout). Figure 4 plots, for the 32 (graph, machine) pairs with both records, the build and peel time of CND summed over the sizes 2 to $s_{\max}$ against the one build of ChainIndex on the same machine. ChainIndex is built faster by 3.1 $\times$ on com-amazon (6 sizes) to 3547 $\times$ on web-it-2004 (431 sizes), median 24 $\times$. The ratio is largest where the sizes are many: on web-BerkStan (201 sizes) CND takes 2.0 hours and ChainIndex 27 seconds, and on web-uk-2005 CND takes 15.7 hours and ChainIndex 18 seconds. Two parts of Section 7 contribute to the ratio: the clique tree is built once and never edited, so it serves every size, and the peel of a size counts cliques only for the vertices whose values are not yet on the floor, the unsettled vertices of Theorem 7.4. Figure 5 compares this peel with the peel of every vertex at every size, against the share of stored values (Table 1), which is small when most pairs are settled. On the 5 graphs that store less than 5 percent of their values, the peel is 15 $\times$ to 126 $\times$ faster and the whole build 1.6 $\times$ to 5.5 $\times$. On the 10 graphs that store more than half, the peel is 1.0 $\times$ to 2.8 $\times$ faster.

The clique tree takes 12 to 64 percent of the build time, the peel 2 to 63 percent, and the index passes 22 to 85 percent.

Exp-2: Index size. Table 1 lists every graph with its number of chains, its vertices per chain and its share of stored values, the pairs (v, s) with a nonzero value whose value is a residue. The next columns are the sizes of ChainIndex and of Baseline with their

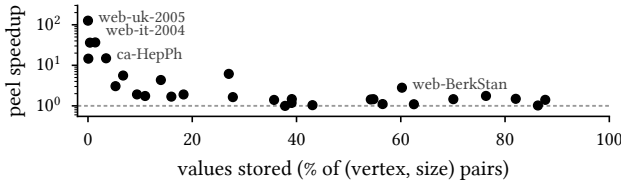


Figure 5: Peel speedup from the settled vertices.

ratio, and the last two the build time and memory usage of Build (Exp-1). ChainIndex is $1.28\times$ to $48.7\times$ smaller than Baseline, median $7.0\times$, and the ratio grows with the number of vertices per chain. email-Eu-core, with 1,005 vertices in 667 chains, 1.5 per chain, is the worst case at $1.28\times$, and web-uk-2005, with 129,632 vertices in 839 chains, 154 per chain, is the best at $48.7\times$. If the graph keeps its original vertex ids, the index also stores the permutation from ids to labels (Section 5.1), n labels, which lowers the ratios to $1.25\times$ to $43.2\times$ (median $4.4\times$).

Exp-3: Size by part. The per-size trees with their run arrays take 35 to 87 percent of the index (median 57), and they are its largest part on 24 of the 29 graphs. The records of the chains (Section 5.4) take 13 to 59 percent (median 36), and the vertex-to-chain map 1 to 14 percent. The records are the larger part on soc-pokec, com-youtube, dblp-coauthor and the two citation graphs. Only the residues are stored (Theorem 5.6), and the share of stored values (Table 1) ranges from 0.0 to 87.7 percent, median 36 percent. web-uk-2005 has 23.5 million pairs and stores one residue in total, since almost every vertex is on its floor from size 2 on, and email-Eu-core stores 87.7 percent of its values. On com-dblp 84,010 of 1,246,933 pairs are residues, and stored once per chain they are 9,737 values.

Exp-4: Design study. We measured the layout decisions one at a time on ca-GrQc, ca-HepPh, com-dblp, web-Stanford and amazon0302. Trees over twin classes (Section 4) do not help, since twin classes have 1.03 to 1.28 vertices on average: they are at best $1.24\times$ smaller than Baseline and on three of the five graphs larger. Trees over chains with an explicit vertex-to-chain map are $2.5\times$ to $4.8\times$ smaller than Baseline. Aligned labels, which remove the map, bring the factor to $3.7\times$ to $8.3\times$, and the runs with their entries, on top of the labels, to $4.4\times$ to $10.2\times$. We also measured the skyline layout of SGL [33] adapted to chains, one entry per chain and size instead of the trees. It took 1.01 to 1.12 times the space of the tree layout and 2.3 to 6.3 times longer to answer an own-level query, so it is not part of the index.

Exp-5: Query latency. Figure 6 reports the community queries of the three levels for both indexes on the 44 (graph, machine) pairs measured together. ChainIndex locates a community in 12 to 65 ns at the own level, median 24 ns. Baseline takes about half, since it reads the own node from the vertex directly. Panel (c) gives the medians per level: 22 ns at the own level, 39 ns at half the level and 18 ns at $k = 1$ for ChainIndex. With the jump pointers of Section 6 no climb costs more than 92 ns. The parent-pointer climb of Baseline costs up to $12\times$ the time of ChainIndex: 216 ns against 19 ns for the climb to the root on dblp-coauthor, whose hierarchy has 5,543 levels. Listing time is the time to write the answer out as vertex labels. It grows linearly with the answer size for both indexes (panel (a)), at 0.05 to 0.5 ns per vertex above a fixed cost

of a few nanoseconds. Baseline pays the fixed cost once, for one memory copy of its block of the array, and ChainIndex once per range, however many labels the range expands to. Panel (b) plots the listing time of Baseline divided by that of ChainIndex against the mean number of vertices per range, so ChainIndex is the faster one above 1. ChainIndex is faster where the answer is a few long ranges, on the social, product and web graphs. It is slower where the answer is split into short ranges: email-Eu-core, cit-HepPh and ca-AstroPh have 2 to 5 vertices per range. Over the 132 (graph, level) points the ratio ranges from 0.1 to 2.8 (median 1.01), and ChainIndex is the faster one on 69 of the 132. ChainIndex answers a value query in 6 to 144 ns, median 18 ns.

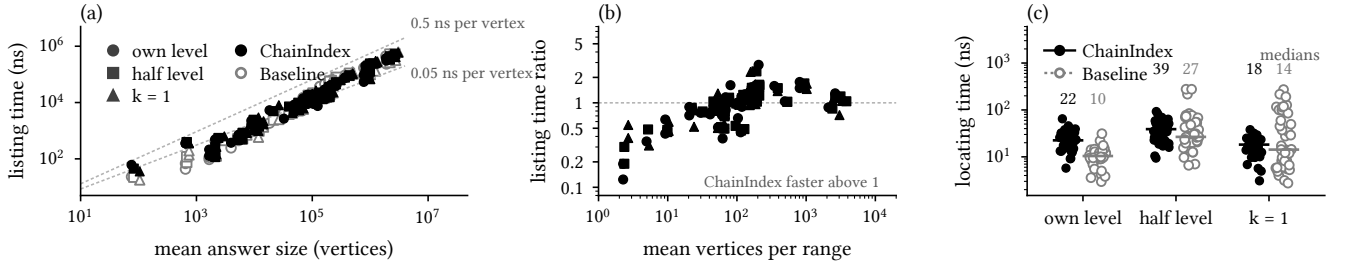
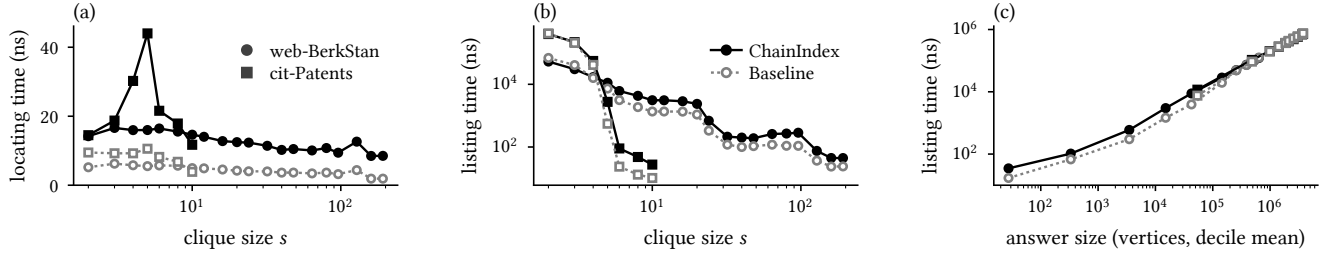
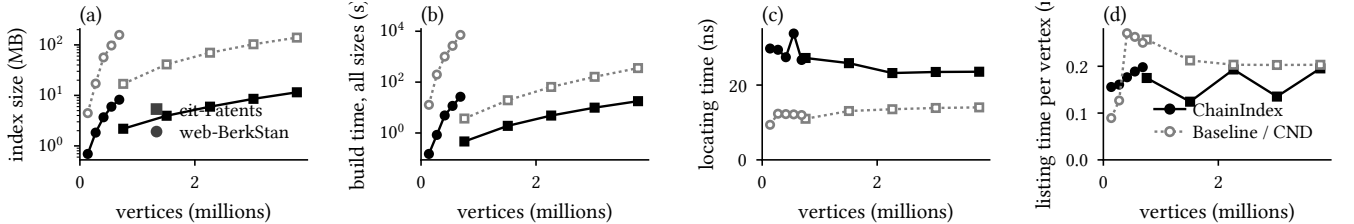
Exp-6: Latency by clique size and by answer size. Figure 7 separates two variables that the workload above mixes, the clique size and the answer size, on two representative graphs measured on server 2, web-BerkStan and cit-Patents. The other graphs show the same trends. Panels (a) and (b) plot the locating and the listing time against the clique size, over 5,000 own-level queries per size s whose vertex is drawn uniformly among the vertices in an s -clique. At $s = 2$ and 3, where the answers are large and consist of a few long runs, ChainIndex lists the answer in 1.06 to 0.74 times the time of Baseline. From $s = 4$ on, where the own-level answers shrink and spread over many ranges, Baseline is faster, up to $5\times$ at $s = 5$ on cit-Patents, where the mean answer has 6,025 vertices in 2,317 ranges. Panel (c) sorts the 20,000 own-level queries of the workload above by answer size into ten groups of equal count and plots the listing time per group. Below 10^5 vertices Baseline is $1.5\times$ to $2.3\times$ faster, between 10^5 and 10^6 the gap closes, and from 10^6 vertices on the two listing times agree within 5 percent.

Exp-7: Scalability. Figure 8 follows cit-Patents and web-BerkStan on server 2 through vertex-induced random samples of 20, 40, 60 and 80 percent of their vertices up to the full graph. The panels show the sizes of both indexes, the build time of ChainIndex and of CND run once per size, and the own-level locating and listing time of both indexes. The samples grow in density as well as in size: the largest cliques of web-BerkStan go from 45 to 202 vertices. ChainIndex grows with the vertices, at 2.9 to 3.2 bytes per vertex on cit-Patents and 5.3 to 12.6 on web-BerkStan, whose cliques grow with the sample, while Baseline grows with the number of (vertex, size) pairs. The ratio of the two sizes therefore rises along each curve, from $7.7\times$ to $12.1\times$ on cit-Patents and from $6.4\times$ to $19.2\times$ on web-BerkStan. The build time grows faster than the sample, from 0.56 to 19.5 seconds as the vertices of cit-Patents grow five times and from 0.21 to 65.6 seconds on web-BerkStan. CND once per size grows faster still (panel (b)). Its build and peel time, its whole run less the reading of the graph, is 6.6 to 18 times the build time of ChainIndex along the cit-Patents curve and 61 to 135 times along the web-BerkStan curve.

Exp-8: Case Study I: the best size per query. Three graphs of the SNAP collection [14] come with ground-truth communities, 5,000 per graph, those of highest quality: com-dblp (publication venues), com-amazon (product categories) and com-youtube (user groups). For a member v of a ground-truth community D and every size $s \leq \omega(v)$ and level k , we score the (s, k) -core of v by its F1 score against D . The F1 score is the harmonic mean of the precision and the recall of the core as a guess of D . The whole (s, k) space

Table 1: Index size against Baseline, with build time and memory usage.

Graph	Type	n	$s_{\max}$	Chains	n/chains	Values stored	ChainIndex (MB)	Baseline (MB)	Ratio	Build (s)	Memory (GB)
email-Eu-core	comm.	1,005	35	667	1.5	87.7%	0.1	0.2	1.3×	0.21	0.01
cit-HepPh	citation	34,546	31	16,623	2.1	86.3%	1.5	3.1	2.0×	0.73	0.10
soc-Epinions1	social	75,879	68	8,821	8.6	54.7%	1.1	3.1	2.9×	8.42	0.31
soc-pokec	social	1,632,803	48	383,206	4.3	82.1%	30.9	97.1	3.1×	64.85	2.83
loc-Brightkite	social	58,228	53	6,326	9.2	56.5%	0.8	2.5	3.2×	0.77	0.08
ca-AstroPh	collab	18,772	57	3,331	5.6	11.0%	0.6	2.1	3.6×	0.51	0.02
tech-as-skitter	comm.	1,694,616	112	163,053	10.4	76.3%	26.8	118	4.4×	693.07	13.74
ca-GrQc	collab	5,242	44	716	7.3	5.3%	0.1	0.3	4.4×	0.02	0.01
amazon0302	product	262,111	7	40,867	6.4	43.1%	2.9	13.7	4.7×	0.33	0.12
com-amazon	product	334,863	7	46,023	7.3	39.1%	3.1	14.8	4.8×	0.83	0.11
amazon-copurchase	product	548,552	7	56,315	9.7	37.8%	3.8	18.8	5.0×	0.35	0.16
soc-Slashdot0902	social	82,168	56	6,798	12.1	62.5%	0.6	2.9	5.0×	1.71	0.24
ca-HepTh	collab	9,877	32	1,104	8.9	16.0%	0.1	0.4	5.5×	0.01	0.01
web-Google	web	875,713	45	73,836	11.9	35.7%	10.6	60.7	5.7×	5.66	0.34
web-Stanford	web	281,903	72	17,963	15.7	54.2%	3.4	23.7	7.0×	4.46	0.23
ca-CondMat	collab	23,133	26	1,917	12.1	9.4%	0.2	1.2	7.3×	0.08	0.01
ca-HepPh	collab	12,008	239	1,136	10.6	3.5%	0.2	1.8	9.8×	0.28	0.02
ca-MathSciNet	collab	332,689	25	15,869	21.0	18.3%	1.2	11.8	10.0×	0.75	0.09
com-dblp	collab	317,080	114	13,459	23.6	6.7%	1.6	15.9	10.2×	1.10	0.11
com-youtube	social	1,134,890	52	42,815	26.5	39.1%	3.0	32.3	10.9×	7.03	0.49
dblp-core30	collab	1,206	114	35	34.5	0.1%	0.0	0.5	11.0×	0.03	0.01
dblp-coauthor	collab	4,049,537	450	363,201	11.1	14.0%	42.2	506	12.0×	81.68	8.02
cit-Patents	citation	3,774,768	65	203,819	18.5	70.1%	11.5	139	12.1×	17.82	1.99
web-it-2004	web	509,338	432	35,402	14.4	1.4%	9.4	130	13.8×	11.16	0.48
wiki-Talk	comm.	2,394,385	132	29,699	80.6	27.8%	3.8	60.3	15.9×	616.99	20.10
web-BerkStan	web	685,230	202	39,559	17.3	60.2%	8.2	158	19.2×	26.53	1.04
web-NotreDame	web	325,729	156	9,122	35.7	27.0%	1.0	23.7	24.7×	1.39	0.17
ca-coauthors-dblp	collab	540,486	337	9,326	58.0	0.4%	4.4	202	45.8×	28.45	0.81
web-uk-2005	web	129,632	500	839	154.5	0.0%	3.9	190	48.7×	18.10	0.28

**Figure 6: Community queries: (a) listing time by answer size and level, (b) listing time of Baseline over that of ChainIndex by vertices per range, (c) locating time by level.****Figure 7: Own-level latency by clique size and by answer size.****Figure 8: Scalability over vertex-induced samples.**

of a query is scanned from the index in 1 to 50 μ s without listing a community. The size of a core comes from its entries. The size of

its intersection with D comes from testing, for every member of D , whether its own node lies in the subtree of the core. Since the

scale of k differs between sizes, we compare the best level of each size. The k -core at its best level, that is, size 2, is the first choice. We compare it with the triangle size $s = 3$ at its best level, the best single size for the graph (by mean F1) at its best level, and the best size and level chosen per query. *No single size is right on com-dblp*. The best size spreads over 2 to 8. Size 2 is best for 26 percent of the queries, size 3 for 23 percent, size 4 for 18, size 5 for 13, and sizes 6 and above for 20. Choosing the size and level per query gives an F1 of 0.179, $3.6\times$ the 0.049 of the k -core and $1.2\times$ the 0.147 of the best fixed size, $s = 5$, at its best level. For 74 percent of the queries, some size other than 2 beats the k -core at every level. *One size can be enough*. On com-amazon the best size is 3 for 59 percent of the queries. The triangle size alone reaches 0.824 against 0.851 for the per-query choice, $1.7\times$ and $1.8\times$ the 0.474 of the k -core. On com-youtube every size is far from the ground-truth groups, with F1 below 0.04. Its groups are known to be sparse [29], and the comparison says nothing there.

Exp-9: Case Study II: Amazon products across sizes. We study com-amazon joined with the SNAP product metadata (titles, groups and category paths), available for 334,852 of its 334,863 products. For a query product q , the *leaf purity* of a community is the fraction of its members whose leaf category, the last category on their path, is that of q . Its *subject purity* is the fraction whose subject, the third level of the category tree such as Jazz or Computers & Internet, is that of q . We follow 5,000 products, drawn uniformly among those that lie in a triangle, through their own-level community at every size. *Purity rises and the communities shrink with the clique size*. At size 2 (the k -core), three quarters of the 5,000 products sit in one core of 157,331 products, with a mean leaf purity of 10 percent. Size 3 brings the median community to 38 products at a mean leaf purity of 44 percent, size 4 to 11 products at 70 percent, and size 5 to 8 products at 79 percent. From size 4 on, the most common subject in a community, whatever the subject of the query, covers 99 percent of its members on average. *The communities look like shelves*. “Kind of Blue” sits at size 2 in a core of 36,878 products, 1 percent of them jazz, smaller than the store above because its level is higher. At size 3 its community is the 23 albums of the Blue Note record label. At size 5 it is six jazz albums: Kind of Blue, The Best of the Blue Note Years, Monk’s Dream, Thelonious Monk with John Coltrane, Mingus Ah Um and Saxophone Colossus. “Introduction to Algorithms” goes from the 157,331-product core to 57 computing books at size 3 and 13 theory textbooks at size 4, Approximation Algorithms and Randomized Algorithms among them. The Godfather soundtrack reaches 28 products at size 5 at a leaf purity of only 32 percent. They still form one shelf, which the category tree has no name for: the four Godfather soundtracks among albums of Italian-American singers such as Jerry Vale and Lou Monte. *Practical guideline*. Size 3 is the first size at which the median community, 38 products, reads as a list. Sizes 4 and 5 bring it to a shelf of 8 to 11 products at 70 to 79 percent leaf purity, each community a few ranges listed in under 50 ns.

9 Related Work

Nucleus decompositions. The clique cores of this paper are the $(1, s)$ -nuclei of the (r, s) -nucleus decomposition, introduced with

the hierarchy of the nuclei of one parameter pair in [22, 23]. Hierarchy construction from the peel order [20] and local [21] and parallel [25, 26] algorithms followed. Clique counting by pivoting [11, 30] counts the cliques of every size from one compact tree without listing them. The counting-based decomposition CND [32] peels one size from an index of clique paths that it rewrites as vertices leave. All of these compute the values, and at most the hierarchy, of one size per run. We store every size’s hierarchy for querying, built from one clique tree that no peel edits and that gives the values and the hierarchy of every size.

Community search and its indexes. Community search returns the cohesive subgraph of a query vertex [4, 6, 27]. Indexes for the k -core model store the core hierarchy as a tree of nested cores, the ShellStruct and CL-tree structures [2, 5]. They answer a query by locating the query vertex’s node and walking up to the requested k . The size-2 layer of our index is such a tree, and Theorem 5.1 makes each of its communities one label range. For the k -truss model, TCP-index [9] keeps a maximum spanning forest per vertex, and EquiTruss [1] partitions the edges into truss-equivalence classes, edges that lie in the same k -truss communities for every k . Chains are the same idea taken across sizes, with the vertex as the unit. Indexes over a second axis, time in historical and temporal core queries [28, 31], store versions of one hierarchy. Our second axis, the clique size, is one along which the values reach a closed form (Theorem 5.6). The SGL index for bipartite graphs [33] builds its index from units that, within one parameter setting, are neither nested nor overlapping, and it retrieves each vertex once per query. Across parameter settings a vertex there belongs to one unit per setting, whereas chains partition the vertices once for every size.

10 Conclusion

In this paper, we presented ChainIndex, an index for the clique-core hierarchies of every clique size at once. The vertices no query can tell apart are exactly the chains, the coarsest lossless partition, and aligned labels, a chain order and run arrays turn the partition into communities located in constant time. On 29 graphs the index is $1.28\times$ to $48.7\times$ smaller than one hierarchy per size (median $7.0\times$). It lists a community about as fast as a memory copy, and, from one static clique tree, it is built for every size $3.1\times$ to $3547\times$ faster than one decomposition per size. Maintaining the index under edge insertions, and the same partition with an edge as the unit, are future work.

References

- [1] Esra Akbas and Peixiang Zhao. 2017. Truss-based community search: a truss-equivalence based indexing approach. *Proc. VLDB Endow.* 10, 11 (Aug. 2017), 1298–1309. doi:10.14778/3137628.3137640
- [2] Nicola Barbieri, Francesco Bonchi, Edoardo Galimberti, and Francesco Gullo. 2015. Efficient and effective community search. *Data Mining and Knowledge Discovery* 29, 5 (2015), 1406–1433.
- [3] Alex Beutel, Wanhong Xu, Venkatesan Guruswami, Christopher Palow, and Christos Faloutsos. 2013. CopyCatch: stopping group attacks by spotting lockstep behavior in social networks. In *Proceedings of the 22nd International Conference on World Wide Web (Rio de Janeiro, Brazil) (WWW ’13)*. Association for Computing Machinery, New York, NY, USA, 119–130. doi:10.1145/2488388.2488400
- [4] Wanyun Cui, Yanghua Xiao, Haixun Wang, and Wei Wang. 2014. Local search of communities in large graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data*. 991–1002.
- [5] Yixiang Fang, Reynold Cheng, Siqiang Luo, and Jiafeng Hu. 2016. Effective Community Search for Large Attributed Graphs. *Proc. VLDB Endow.* 9, 12 (2016), 1233–1244.

- [6] Yixiang Fang, Xin Huang, Lu Qin, Ying Zhang, Wenjie Zhang, Reynold Cheng, and Xuemin Lin. 2020. A survey of community search over big graphs. *The VLDB Journal* 29, 1 (2020), 353–392.
- [7] Santo Fortunato. 2010. Community detection in graphs. *Physics Reports* 486, 3–5 (2010), 75–174. doi:10.1016/j.physrep.2009.11.002
- [8] Bryan Hooi, Hyun Ah Song, Alex Beutel, Neil Shah, Kijung Shin, and Christos Faloutsos. 2016. FRAUDAR: Bounding Graph Fraud in the Face of Camouflage. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (San Francisco, California, USA) (KDD '16). Association for Computing Machinery, New York, NY, USA, 895–904. doi:10.1145/2939672.2939747
- [9] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-truss community in large and dynamic graphs. In *Proceedings of the 2014 ACM SIGMOD International Conference on Management of Data* (Snowbird, Utah, USA) (SIGMOD '14). Association for Computing Machinery, New York, NY, USA, 1311–1322. doi:10.1145/2588555.2610495
- [10] Guy Jacobson. 1989. Space-efficient Static Trees and Graphs. In *30th Annual Symposium on Foundations of Computer Science (FOCS)*. 549–554. doi:10.1109/SFCS.1989.63533
- [11] Shweta Jain and C. Seshadhri. 2020. The Power of Pivoting for Exact Clique Counting. In *Proceedings of the 13th International Conference on Web Search and Data Mining* (Houston, TX, USA) (WSDM '20). Association for Computing Machinery, New York, NY, USA, 268–276. doi:10.1145/3336191.3371839
- [12] Gyula O. H. Katona. 1968. A Theorem of Finite Sets. In *Theory of Graphs*. Akadémiai Kiadó, Budapest, 187–207.
- [13] Joseph B. Kruskal. 1963. The Number of Simplices in a Complex. In *Mathematical Optimization Techniques*, Richard Bellman (Ed.). University of California Press, Berkeley, 251–278.
- [14] Jure Leskovec and Rok Sosič. 2016. SNAP: A general-purpose network analysis and graph-mining library. *ACM Transactions on Intelligent Systems and Technology* 8, 1 (2016), 1.
- [15] Greg Linden, Brent Smith, and Jeremy York. 2003. Amazon.com Recommendations: Item-to-Item Collaborative Filtering. *IEEE Internet Computing* 7, 1 (Jan. 2003), 76–80. doi:10.1109/MIC.2003.1167344
- [16] László Lovász. 1979. *Combinatorial Problems and Exercises*. North-Holland, Amsterdam.
- [17] Eugene W. Myers. 1983. An Applicative Random-Access Stack. *Inform. Process. Lett.* 17, 5 (1983), 241–248.
- [18] M. E. J. Newman. 2001. The structure of scientific collaboration networks. *Proceedings of the National Academy of Sciences* 98, 2 (2001), 404–409. doi:10.1073/pnas.98.2.404
- [19] Branco Ponomariov and Craig Boardman. 2016. What is co-authorship? *Scientometrics* 109, 3 (2016), 1939–1963. doi:10.1007/s11192-016-2127-7
- [20] Ahmet Erdem Sariyüce and Ali Pinar. 2016. Fast hierarchy construction for dense subgraphs. *Proc. VLDB Endow.* 10, 3 (Nov. 2016), 97–108. doi:10.14778/3021924.3021927
- [21] Ahmet Erdem Sariyüce, C. Seshadhri, and Ali Pinar. 2018. Local algorithms for hierarchical dense subgraph discovery. *Proc. VLDB Endow.* 12, 1 (Sept. 2018), 43–56. doi:10.14778/3275536.3275540
- [22] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2015. Finding the Hierarchy of Dense Subgraphs using Nucleus Decompositions. In *Proceedings of the 24th International Conference on World Wide Web* (Florence, Italy) (WWW '15). International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 927–937. doi:10.1145/2736277.2741640
- [23] Ahmet Erdem Sariyüce, C. Seshadhri, Ali Pinar, and Umit V. Catalyurek. 2017. Nucleus Decompositions for Identifying Hierarchy of Dense Subgraphs. *ACM Trans. Web* 11, 3, Article 16 (July 2017), 27 pages. doi:10.1145/3057742
- [24] Stephen B. Seidman. 1983. Network structure and minimum degree. *Social Networks* 5, 3 (1983), 269–287. doi:10.1016/0378-8733(83)90028-X
- [25] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2021. Theoretically and practically efficient parallel nucleus decomposition. *Proc. VLDB Endow.* 15, 3 (Nov. 2021), 583–596. doi:10.14778/3494124.3494140
- [26] Jessica Shi, Laxman Dhulipala, and Julian Shun. 2024. Parallel Algorithms for Hierarchical Nucleus Decomposition. *Proc. ACM Manag. Data* 2, 1, Article 32 (March 2024), 27 pages. doi:10.1145/3639287
- [27] Mauro Sozio and Aristides Gionis. 2010. The Community-search Problem and How to Plan a Successful Cocktail Party. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (KDD). doi:10.1145/1835804.1835923
- [28] Anxin Tian, Alexander Zhou, Yue Wang, Xun Jian, and Lei Chen. 2024. Efficient Index for Temporal Core Queries over Bipartite Graphs. *Proc. VLDB Endow.* 17, 11 (July 2024), 2813–2825. doi:10.14778/3681954.3681965
- [29] Jaewon Yang and Jure Leskovec. 2015. Defining and evaluating network communities based on ground-truth. *Knowledge and Information Systems* 42, 1 (2015), 181–213. doi:10.1007/s10115-013-0693-z
- [30] Xiaowei Ye, Rong-Hua Li, Qiangqiang Dai, Hongzhi Chen, and Guoren Wang. 2022. Lightning Fast and Space Efficient k-clique Counting. In *Proceedings of the ACM Web Conference 2022* (Virtual Event, Lyon, France) (WWW '22). Association for Computing Machinery, New York, NY, USA, 1191–1202. doi:10.1145/3485447.3512167
- [31] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On Querying Historical K-Cores. *Proc. VLDB Endow.* 14, 11 (2021), 2033–2045. doi:10.14778/3476249.3476260
- [32] Wenqian Zhang, Zhengyi Yang, Dong Wen, Yi Ding, Wenjie Zhang, and Xuemin Lin. 2026. Nucleus Decomposition Revisited: An Efficient Counting-Based Approach. *Proc. ACM Manag. Data* 4, 3, Article 215 (May 2026), 26 pages. doi:10.1145/3802092
- [33] Chuanyu Zong, Lingbiao Meng, Xiaochun Yang, Bin Wang, Tao Qiu, and Rui Zhu. 2026. Zero-Redundancy Search for Bi-Components in Bipartite Graphs. *Proc. ACM Manag. Data (SIGMOD)* (2026). Article 3802129.